Concurrency control

Concurrency control

- Concurrency controlling techniques ensure that multiple transactions are executed simultaneously while maintaining the ACID properties of the transactions.

Concurrency control

- Simultaneous execution of transactions over a shared database can create several data integrity and consistency problems.
  - Lost Update
  - Dirty Read Problems
  - Incorrect Summary Problems
  - Unrepeatable Read Problem

Concurrency control

- **Conflicting operations:** Two operations are called as conflicting operations if all the following conditions hold true for them
  - Both the operations belong to different transactions
  - Both the operations are on same data item
  - Read – Read (No conflict)
  - Read – Write (or Write – Read)Conflict!
  - Write – Write (Conflict)

Concurrency control

- W(X) of transaction T1 and R(X) of transaction T2
- W(X) of T1 and W(X) of T2
- W(X) of T1 and W(Y) of T2
- R(X) of T1 and R(X) of T2
- W(X) of T1 and R(X) of T1

PROBLEM 1: LOST UPDATE PROBLEM

- The problem occurs when two different database transactions perform the read/write operations on the same database items in an interleaved manner (i.e., concurrent execution) that makes the values of the items incorrect hence making the database inconsistent.
- **For example:**
- **Consider the below diagram where two transactions T_X and T_Y, are performed on the same account A where the balance of account A is $300.**

Time	T_x	T_y
t_1	READ (A)	—
t_2	A = A - 50	
t_3	—	READ (A)
t_4	—	A = A + 100
t_5	—	—
t_6	WRITE (A)	—
t_7		WRITE (A)

LOST UPDATE PROBLEM

- At time t1, transaction T_X reads the value of account A, i.e., $300 (only read).
- At time t2, transaction T_X deducts $50 from account A that becomes $250 (only deducted and not updated/write).
- Alternately, at time t3, transaction T_Y reads the value of account A that will be $300 only because T_X didn't update the value yet.
- At time t4, transaction T_Y adds $100 to account A that becomes $400 (only added but not updated/write).
- At time t6, transaction T_X writes the value of account A that will be updated as $250 only, as T_Y didn't update the value yet.
- Similarly, at time t7, transaction T_Y writes the values of account A, so it will write as done at time t4 that will be $400. It means the value written by T_X is lost, i.e., $250 is lost.
- Hence data becomes incorrect, and database sets to inconsistent.

THE LOST UPDATE PROBLEM

Transaction T:		Transaction U:	
balance = b.getBalance();		*balance = b.getBalance();*	
*b.setBalance(balance*1.1);*		*b.setBalance(balance*1.1);*	
a.withdraw(balance/10)		*c.withdraw(balance/10)*	
balance = b.getBalance();	$200		
		balance = b.getBalance();	$200
		*b.setBalance(balance*1.1);*	$220
*b.setBalance(balance*1.1);*	$220		
a.withdraw(balance/10)	$80		
		c.withdraw(balance/10)	$280

Figure 13.5

- the initial balances of accounts A, B, C are $100, $200. $300
- both transfer transactions increase B's balance by 10%

- the net effect should be to increase *B* by 10%
- twice - 200, 220, 242.
- but it only gets to 220. *U*'s update is lost.

THE INCONSISTENT RETRIEVALS PROBLEM

- Inconsistent retrievals occur when a retrieval transaction observes values that are involved in an ongoing updating transaction.
- Consider a situation, where one transaction is applying the aggregate function on some records while another transaction is updating these records.
- The aggregate function may calculate some values before the values have been updated and others after they are updated.

THE INCONSISTENT RETRIEVALS PROBLEM

Transaction *V*: *a.withdraw(100)* *b.deposit(100)*		Transaction *W*: *aBranch.branchTotal()*	
a.withdraw(100);	$100		
		total = a.getBalance()	$100
		total = total+b.getBalance()	$300
		total = total+c.getBalance()	
b.deposit(100)	$300	:	

Figure 13.6

- the initial balances of accounts A and B are both $200
- *V* transfers $100 from *A* to *B* while *W* calculates branch total (which should be $400)

- we see an inconsistent retrieval because *V* has only done the withdraw part when *W* sums balances of *A* and *B*

UNREPEATABLE READ PROBLEM

- That occurs when in a transaction, two different values are read for the same database item.
- **For example:**
- **Consider two transactions, T_X and T_Y, performing the read/write operations on account A, having an available balance = $300. The diagram is shown below:**

Time	T_x	T_y
t_1	READ (A)	—
t_2	—	READ (A)
t_3	—	A = A + 100
t_4	—	WRITE (A)
t_5	READ (A)	—

UNREPEATABLE READ PROBLEM

- At time t1, transaction T_X reads the value from account A, i.e., $300.
- At time t2, transaction T_Y reads the value from account A, i.e., $300.
- At time t3, transaction T_Y updates the value of account A by adding $100 to the available balance, and then it becomes $400.
- At time t4, transaction T_Y writes the updated value, i.e., $400.
- After that, at time t5, transaction T_X reads the available value of account A, and that will be read as $400.
- It means that within the same transaction T_X, it reads two different values of account A, i.e., $ 300 initially, and after updation made by transaction T_Y, it reads $400. It is an unrepeatable read and is therefore known as the Unrepeatable read problem.
- Thus, in order to maintain consistency in the database and avoid such problems that take place in concurrent execution, management is needed, and that is where the concept of Concurrency Control comes into role.